

Security

Transport Security

All API communication must use **HTTPS** with a minimum of **TLS 1.2**. Connections using TLS 1.0 or 1.1 will be rejected. TLS 1.3 is recommended where supported.

Authorization

All requests from the Integration Service include the following authorization headers. The Operator must validate these on every incoming request.



Note

The operator is identified exclusively via the `X-Operator-Id` header. There is no `operator` field in the request body.

Authorization Headers

Header	Type	Example Value	Notes
X-Operator-Id	string	optimo-main	Operator identifier, constant per operator/environment. Sole mechanism for identifying the calling operator.
X-Timestamp	int64	1730379235	Unix timestamp (seconds), ±30s drift allowed
X-Nonce	string	b7a9d3e0f4124b6f9c1a	Random per request; must not repeat

X-Sign	string	a4f1b2c8d 9e0f11223 3445566	Hex-encoded HMAC-SHA1 signature
--------	--------	-----------------------------------	------------------------------------



Integrator Responsibilities

- Ensure **X-Timestamp** is generated from a reliable time source (NTP-synced servers recommended).
- Use **cryptographically secure random generator** for X-Nonce.
- Reject requests with:
 - Timestamp outside allowed window ($\pm 30s$)
 - Missing/invalid X-Sign
 - Reused X-Nonce (protects from replay attacks)

Replay Protection

Request validation must follow this order:

1. **Timestamp check** — reject if `X-Timestamp` is outside the $\pm 30s$ drift window.
2. **Nonce check** — reject if `X-Nonce` was already seen within the current drift window.
3. **Signature check** — verify `X-Sign`. If verification fails, return `result_code: 1001` (Invalid signature).

Nonces only need to be stored for the duration of the timestamp drift window (**30 seconds TTL**). Any replayed request older than 30s is already rejected at step 1, so there is no need to store nonces indefinitely. A TTL-based cache (e.g. Redis with 30s expiry) is sufficient.

X-Sign Calculation

1. Collect all request body fields **except** `attributes`.
2. Add the authorization headers: `X-Operator-Id`, `X-Timestamp`, `X-Nonce`.
3. Sort all entries alphabetically by key.
4. Build a query string: `key1=value1&key2=value2&...`
5. URL-encode the entire query string.

6. Sign using **HMAC-SHA1** with Operator Key.



Important

Field keys used in the signature match the JSON field names exactly as they appear in the serialized request body (e.g. `user_id`, `game_id`, `bet_type`, `session_id`). The `attributes` object is always excluded from signature calculation.

Amount Formatting in Signatures



Common Integration Bug

The most frequent cause of signature mismatch is inconsistent amount formatting. The `amount` value used in the signature string **must exactly match** the value in the JSON body — including decimal places.

The JSON body may contain any valid numeric representation (`1` , `1.0` , `1.00` are all valid JSON). However, when building the signing string, the operator **must normalize** the amount to the currency's **precision scale** (see [API Protocols — Amount Precision](#)):

- USD, EUR, GEL: **2 decimal places**
- Other currencies: precision provided during integration

Body value	Signing value	Result
<code>"amount": 1</code>	<code>amount=1.00</code>	Correct
<code>"amount": 1</code>	<code>amount=1</code>	Signature mismatch
<code>"amount": 1.0</code>	<code>amount=1.00</code>	Correct
<code>"amount": 1.0</code>	<code>amount=1.0</code>	Signature mismatch
<code>"amount": 10.5</code>	<code>amount=10.50</code>	Correct
<code>"amount": 10.5</code>	<code>amount=10.5</code>	Signature mismatch

Always normalize amounts to the full precision scale before building the signing string. For example, for a USD bet of 1 dollar: the JSON body may contain `"amount": 1` , but the signing string must use `amount=1.00` .

Example

Example 1 — Balance request (no amount field):

```
POST /seamless/balance
{
  "user_id": "1234",
  "currency": "USD",
  "game_id": "king_move",
  "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
```



```
}
```

Headers sent:

```
X-Operator-Id: optimo-main  
X-Timestamp: 1730379235  
X-Nonce: b7a9d3e0f4124b6f9c1a
```

**Step 1 — Sorted key-value pairs (body fields + auth headers):**

```
X-Nonce: b7a9d3e0f4124b6f9c1a  
X-Operator-Id: optimo-main  
X-Timestamp: 1730379235  
currency: USD  
game_id: king_move  
session_id: f47ac10b-58cc-4372-a567-0e02b2c3d479  
user_id: 1234
```

**Step 2 — Query string:**

```
X-Nonce=b7a9d3e0f4124b6f9c1a&X-Operator-Id=optimo-main&X-  
Timestamp=1730379235&currency=USD&game_id=king_move&session_id=f47ac10b-  
58cc-4372-a567-0e02b2c3d479&user_id=1234
```

**Step 3 — URL-encoded:**

```
X-Nonce%3Db7a9d3e0f4124b6f9c1a%26X-Operator-Id%3Doptimo-main%26X-  
Timestamp%3D1730379235%26currency%3DUSD%26game_id%3Dking_move%26session_i  
d%3Df47ac10b-58cc-4372-a567-0e02b2c3d479%26user_id%3D1234
```

**Step 4 — HMAC-SHA1 signature:**

```
X-Sign = HMAC-SHA1(url_encoded_string, OPERATOR_KEY)
```

**Example 2 — Bet request (with amount field):**

```
POST /seamless/bet  
{  
  "timestamp": 1730379235000,  
  "bet_type": "BET",  
  "user_id": "1234",  
  "amount": 1.00,  
  "currency": "USD",  
  "game_id": "king_move",  
  "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",  
  "transaction_id": "B112233",  
}
```



```
"round_id": "SBR001",  
"attributes": {  
  "test1": "test value1"  
}  
}
```

Step 1 — Sorted key-value pairs (`attributes` excluded):

```
X-Nonce: b7a9d3e0f4124b6f9c1a  
X-Operator-Id: optimo-main  
X-Timestamp: 1730379235  
amount: 1.00  
bet_type: BET  
currency: USD  
game_id: king_move  
round_id: SBR001  
session_id: f47ac10b-58cc-4372-a567-0e02b2c3d479  
timestamp: 1730379235000  
transaction_id: B112233  
user_id: 1234
```



Note: normalize `amount` to `1.00` in the signing string

The JSON body may contain `"amount": 1`, but the signing string must always use the normalized value `amount=1.00` (USD = 2 decimal places). Using `1` or `1.0` instead of `1.00` in the signing string will produce a different signature.

Step 2 — Query string:

```
X-Nonce=b7a9d3e0f4124b6f9c1a&X-Operator-Id=optimo-main&X-  
Timestamp=1730379235&amount=1.00&bet_type=BET&currency=USD&game_id=king_m  
ove&round_id=SBR001&session_id=f47ac10b-58cc-4372-a567-  
0e02b2c3d479&timestamp=1730379235000&transaction_id=B112233&user_id=1234
```

Step 3 — URL-encode, then Step 4 — HMAC-SHA1 as above.